

# Reproducibility Report on *Distilled Datamodel with Reverse Gradient Matching*

Jingwen Ye Ruonan Yu Songhua Liu Xinchao Wang  
National University of Singapore  
jingweny@nus.edu.sg, xinchao@nus.edu.sg

February 20, 2025

## Abstract

The paper "Distilled Datamodel with Reverse Gradient Matching" introduces a novel data attribution framework leveraging reverse gradient matching to efficiently approximate the influence of training data on a target model while significantly reducing the computational cost of traditional leave-one-out retraining. For reproduction, we followed the methodology outlined in the paper. Yet, the process encountered challenges due to high computational demands and the lack of publicly available pre-trained models. Despite these hurdles, our reproduction confirmed the effectiveness of reverse gradient matching in improving training data attribution efficiency. While the core findings were validated, minor variations in influence estimation accuracy and training efficiency suggest that refinements in experimental setups are necessary. Additionally, the lack of detailed fine-tuning steps, architecture specifications, and training configurations in the original paper posed reproducibility challenges. Our findings demonstrate that distilled "synsets" can efficiently approximate data influence while maintaining high accuracy. However, greater clarity in fine-tuning strategies and model hyperparameters would enhance transparency and ease reproduction. The complete re-implementation, dataset processing, and results of this study will be available [Here].

## 1 Introduction

Understanding the influence of individual training data points on a model's behavior for improving model interpretability, fairness, robustness, and unlearning in machine learning frameworks [7, 9, 10]. Despite numerous approaches to explaining models' decision-making processes, the majority attempt to determine feature relevance and explaining models' internal representations [1, 3, 5, 12]. The challenge in training data attribution involves balancing computational demands with effectiveness. Techniques like influence approximation [4, 6] prioritize efficiency but may lack reliability in non-convex settings. Concurrent research [8, 11] has advanced in approximating the impact of removing small subsets of data on model performance, yet these methods lack flexibility for broader applications and are computationally impractical for large-scale models due to excessive resource requirements.

The paper "Distilled Datamodel with Reverse Gradient Matching" introduces the Distilled Datamodel (DDM) framework, utilizing reverse gradient matching to efficiently approximate training data influence without full model retraining. By distilling the training data into a synthetic representation called a 'synset', this approach becomes scalable for training data attribution, privacy-preserving unlearning, and model diagnostics [13].

The primary objective is to develop a computationally efficient method to analyze how training data influences model learning. The authors propose a two-stage framework:

1. **Offline Training:** Use reverse gradient matching to learn a small, synthetic dataset (synset) that captures the impact of the training data.

2. **Online Evaluation:** Utilize the synset to predict the effects of the removal of individual data points on model performance.

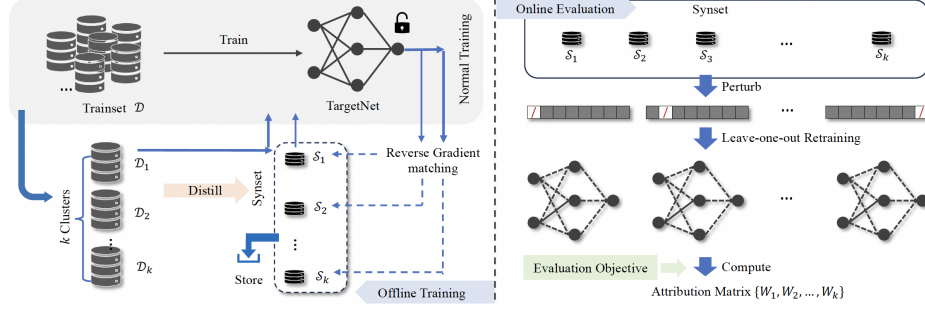


Figure 1: The framework of the proposed distilled datamodel. The synset is distilled during the target network’s regular training process for offline training. Regarding the online assessment, we quickly learn the perturbed model set by perturbing the learned synset, and the result is calculated to create the final attribution matrix. [13]

This approach substantially reduces computational overhead compared to conventional gradient-based methods with no loss of accuracy in identifying critical training data points.

### 1.1 Background and Motivation

To fully comprehend and reproduce the paper’s methodology, familiarity with several key concepts is necessary:

- **Training Data Attribution:** Determining how each training sample affects the performance of the model [13].
- **Gradient Matching:** Gradient matching is a technique for effectively reproducing training effects by matching model updates with synthetic data [15].
- **Dataset Distillation:** Condensing large datasets into smaller synthetic subsets that retain essential learning properties [14].
- **Machine Unlearning:** This technique, which is essential for improving security and privacy, involves eliminating particular training data impacts from a model without requiring a full retraining [2].

This paper has been selected to reproduce because of its novel direction in large-scale training data attribution and efficient machine unlearning. With growing interest in AI privacy, fairness, and explainability, the ability to successfully examine and fine-tune a model’s dependency on certain training data is essential. Furthermore, the computational efficiency of the proposed method makes it highly amenable to practical AI applications. The reproduction project aims to confirm the paper’s primary claims through:

- Implementing the reverse gradient matching algorithm outlined above.
- Evaluating the framework using benchmark datasets such as MNIST to ensure training data attribution.
- A comparison of the performance and accuracy of the suggested method with traditional training data impact estimation methods.

The goal of this project is to evaluate the feasibility, computational complexity, and practical efficiency of the Distilled Datamodel framework.

## 2 Methodology

### 2.1 Mathematical Formulation and Pseudocode

The offline training phase of the Distilled Datamodel (DDM) framework aims to efficiently distill the influence of the training data  $D$  into a much smaller synthetic dataset  $S$  (where  $|S| \ll |D|$ ). First, we cluster  $D$  into  $K$  groups, denoted as  $\{D_1, D_2, \dots, D_K\}$ , since individual data points have minimal influence on the overall behavior of the target network  $M$ .

The target network  $M$  is initialized with parameters  $\theta_0$  and trained on  $D$  for  $\tau$  epochs. The synset  $S$  is then used for fine-tuning  $M$  for an additional  $T$  epochs, resulting in updated parameters  $\theta_\tau$  and  $\tilde{\theta}_T$ .

The synset  $S$  is optimized by minimizing the distribution distance between the fine-tuned model parameters  $\tilde{\theta}_{\kappa T}$  and the unlearned parameters  $\theta_{\kappa \tau}$ :

$$\text{Dist}(\theta_{\kappa \tau}, \tilde{\theta}_{\kappa T}) \quad (1)$$

Gradient descent updates over  $\tau$  and  $T$  epochs are performed as:

$$\theta_{t+1} \leftarrow \theta_t - \eta_a \nabla L(\theta_t, D) \quad (2)$$

$$\tilde{\theta}_{\kappa t+1} \leftarrow \tilde{\theta}_{\kappa t} - \eta_f \nabla L(\tilde{\theta}_{\kappa t}, S_\kappa) \quad (3)$$

Our objective is to ensure that the parameters after unlearning and fine-tuning are approximately equivalent:

$$-\sum_t \nabla L(\tilde{\theta}_{\kappa t}, S_\kappa) = \sum_t \nabla L(\theta_{\kappa t}, D_\kappa) \quad (4)$$

The reverse gradient matching process optimizes the synset  $S$  to predict the parameters of  $M$  that effectively "unlearn" specific data clusters  $D_\kappa$ . This is achieved by minimizing:

$$\arg \min_{|S|=K \times \text{ipc}} \sum_t \sum_\kappa \text{Dist}(-\nabla L(\theta_{\tau-t}, S_\kappa), \nabla L(\theta_t, D_\kappa)) \quad (5)$$

During the online evaluation, both the synset  $S$  and the target network  $M$  are available, allowing for specific model behavior evaluation. The primary concept is leave-one-out cross-validation, systematically perturbing  $D$  by removing training samples to analyze their impact.

For instance, given a test sample  $\{x_t, y_t\}$ , the corresponding prediction result is  $\tilde{y}_t$ , with the target model trained on the full dataset  $D$ . The goal is to establish a relationship between the network's prediction  $\tilde{y}_t$  and  $D$  using an attribution matrix  $W$ :

$$\tilde{y}_t(p_t \circ D) = W \cdot p_t + b, \quad p_t \subseteq \{0, 1\}^K. \quad (6)$$

Here,  $p_t(\kappa) = 0$  denotes the deletion of  $D_\kappa$ . The attribution matrix  $W$  is computed as:

$$\arg \min_W \sum_{p_t \subseteq P_t} \beta_{p_t} \cdot \text{Dist}(\tilde{y}_t(p_t \circ D), W \cdot p_t). \quad (7)$$

Since training  $|P_t|$  new models is computationally expensive, we leverage the synset  $S_\kappa$  for each  $D_\kappa$  to accelerate unlearning.

Below is the pseudocode for the proposed DDM framework.

---

**Algorithm 1** The Proposed DDM Framework

---

```
1: // Offline Training Phase
2: Input:  $D$  (training set),  $M$  (target model),  $\{\theta_0, \theta_1, \dots, \theta_\tau\}$  (training trajectory),  $s$  (trajectory step)
3: Divide  $D$  into  $K$  clusters:  $\{D_1, D_2, \dots, D_K\}$ 
4: Randomly initialize  $K$  synthetic samples to form  $S$ 
5: for each distillation step do
6:   Choose a random start:  $\theta_t$  where  $0 \leq t < \tau$ 
7:   Choose an endpoint:  $\theta_{t+s}$  with  $t + s < \tau$ 
8:   for  $\kappa = 1, 2, \dots, K$  do
9:     Compute gradient on real data:  $\nabla L(\theta_t; D_\kappa)$ 
10:    Compute gradient on synthetic data:  $\nabla L(\theta_{t+s}; S_\kappa)$ 
11:    Update  $S_\kappa$  to minimize  $\text{minDist}(\nabla L(\theta_t; D_\kappa), -\nabla L(\theta_{t+s}; S_\kappa))$ 
12:  end for
13: end for
14: Output: Cluster-wise synthetic data  $\{S_1, S_2, \dots, S_K\}$ 

15: // Online Evaluation Phase
16: Input:  $M$  (target model),  $S$  (synset),  $x_t$  (test sample)
17: Randomly sample perturbations  $p_t$  to form  $P_t$ 
18: for each  $p_t \in P_t$  do
19:   Apply perturbation  $p_t$  on  $S$ :  $p_t \circ S$ 
20:   Fine-tune  $M$  with  $p_t \circ S$ 
21:   Compute output  $\tilde{y}_t$  for  $x_t$ 
22: end for
23: Calculate the attribution matrix  $W$  (using Eq. (7))
24: Output: Attribution matrix  $W$ 
```

---

## 2.2 Experimental Setup

The experimental setup is as follows:

- **Hardware and Software Environment:**
  - CPU: Intel Xeon; GPU: NVIDIA Tesla T4
  - Operating System: Ubuntu 20.04; Programming Language: Python 3.10; Framework: PyTorch 2.0.1
- **Dataset Preparation and Preprocessing:**
  - **Dataset:** MNIST
  - **Preprocessing:** Normalization
- **Choice of Hyperparameters:**
  - Learning rate for synthetic image updates:  $\eta = 10$
  - Learning rate for network parameters:  $\eta = 0.01$
  - Number of clusters per class: 10 (yielding  $K = 100$  clusters for MNIST)
  - Number of distillation steps: 350 and no instruction about this step in the original paper.

### 3 Results

In this section, we provide a detailed comparison of our reproduction attempt with the original paper results. We begin by describing training on the full MNIST dataset with a typical ConvNet. Then, we describe the generation of the synthetic dataset (synset) through reverse gradient matching and compare the reproduced synthetic images to the originals. Lastly, we compare model performance on real and synthetic data.

#### 3.1 Training on the Full MNIST Dataset

We trained a ConvNet on the full MNIST dataset (60,000 training images and 10,000 test images). Since there is no specific mention of hyperparameters and architecture in the original paper, we used a simple ConvNet architecture. Table 1 and Table 2 are the summary of the ConvNet architecture used for training, along with the key training parameters.

| Layer                                    | Output Shape | Parameters |
|------------------------------------------|--------------|------------|
| Conv2d(1, 32, kernel_size=3, padding=1)  | [32, 28, 28] | 320        |
| ReLU                                     | [32, 28, 28] | –          |
| MaxPool2d(2, 2)                          | [32, 14, 14] | –          |
| Conv2d(32, 64, kernel_size=3, padding=1) | [64, 14, 14] | 18,496     |
| ReLU                                     | [64, 14, 14] | –          |
| MaxPool2d(2, 2)                          | [64, 7, 7]   | –          |
| Flatten                                  | [3136]       | –          |
| Linear(3136, 128)                        | [128]        | 401,536    |
| ReLU                                     | [128]        | –          |
| Linear(128, 10)                          | [10]         | 1,290      |
| <b>Total: 421,642 Parameters</b>         |              |            |

Table 1: Summary of the ConvNet architecture used for MNIST.

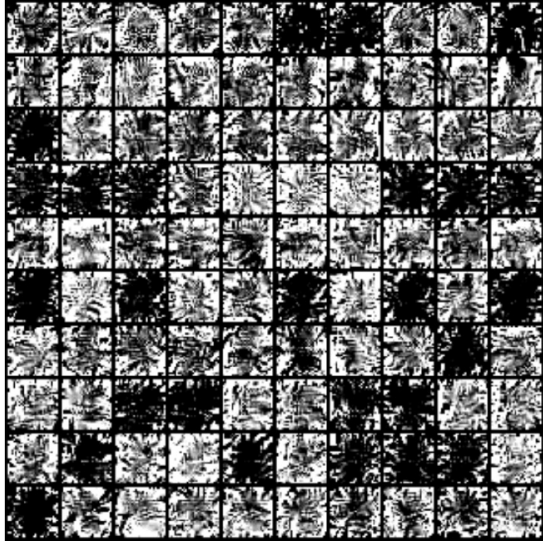
| Training Parameter  | Value                                      |
|---------------------|--------------------------------------------|
| Optimizer           | SGD (learning rate = 0.01, momentum = 0.9) |
| Loss Function       | CrossEntropyLoss                           |
| Number of Epochs    | 20                                         |
| Batch Size          | 64                                         |
| Device              | GPU (Tesla T4)                             |
| Total Training Time | 273.58 seconds                             |

Table 2: Training parameters used for the MNIST experiment.

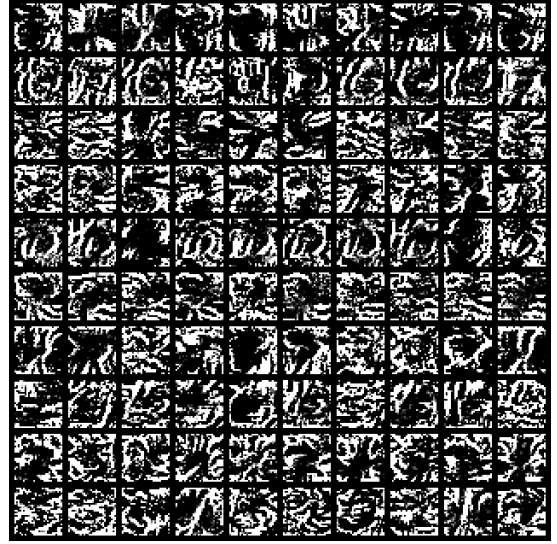
#### 3.2 Synset Generation using Reverse Gradient Matching

To create the synset, we initially carried out class-wise clustering of the MNIST data by splitting each class into 10 clusters (resulting in 100 clusters overall). For each cluster, a single synthetic image was generated using reverse gradient matching. Not only does this technique capture the impact of the training data but hides explicit digit patterns, thus improving privacy.

Figure 2 presents side-by-side comparisons of the synthetic images generated. The left panel shows an example synthetic image from the paper, and the right panel indicates our reproduced synthetic image.



(a) Reverse Gradient Matching (Paper)



(b) Reverse Gradient Matching (Reproduced)

Figure 2: Reverse Gradient Matching. (a) shows original paper synset output and (b) shows reproduced synset output.

### 3.3 Model Performance Comparison

We evaluated the effect of synthetic data training on model performance by contrasting models trained on real data with models trained (or fine-tuned) on distilled synthetic data. Table 4 summarizes the results reported in the original paper alongside our experimental findings.

In the original paper, the synthetic dataset was used in two modes:

- **Scratch Model (Synthetic Data):** Training solely from scratch on the synthetic data resulted in a very low test accuracy of 0.6%, indicating that while the synset captures critical gradient information, it lacks the diversity required to bootstrap learning.
- **Fine-Tuned Model (Synthetic Data):** When a pre-trained network was fine-tuned with the synthetic data, the final test accuracy reached 95.7% .

Our experiments also yield some further information, such as measurements of training times. In particular, when training the original model on actual MNIST data, we obtained a test accuracy of 99.35% with a training time of 273.58 seconds. On the contrary, from-scratch training on synthetic data had a test accuracy of 10.37% on the real test dataset, whereas fine-tuning a pre-trained network on synthetic data had a test accuracy of 83.25% with a further training time of just 0.04 seconds.

These results highlight two important findings:

1. Training exclusively on the distilled synthetic data leads to poor performance due to the lack of full dataset diversity. The synset, although efficient for fine-tuning, cannot substitute for real data when training from scratch.
2. On fine-tuning, the synthetic data facilitates quick adaptation cutting down training time drastically without hindering the model from achieving high performance again. The small disparity in fine-tuning accuracy (83.5% in our experiment compared to 95.7% in the original paper) is possibly due to variations in hyperparameter values, model initialization, or preprocessing methods. The fine-tuning experiment setup is summarized in Table 3

| Parameter        | Value                             |
|------------------|-----------------------------------|
| Model            | Fine-Tuned Model (Synthetic Data) |
| Number of Epochs | 3                                 |
| Learning Rate    | 0.005                             |
| Optimizer        | SGD (momentum = 0.9)              |
| Batch Size       | 16                                |
| Device           | GPU (Tesla T4)                    |

Table 3: Fine-tuning experiment setup for the synthetic dataset. The original model (pre-trained on real MNIST data) was fine-tuned on the distilled synthetic data using 3 epochs of training with SGD (lr = 0.005, momentum = 0.9).

Table 4 summarizes the performance metrics from both the original paper and our experimental findings, including the training time measurements that were not reported in the original work.

| Original Paper                    |               |                     |
|-----------------------------------|---------------|---------------------|
| Model / Data                      | Test Accuracy | Training Time (sec) |
| Original Model (Real Data)        | —             | —                   |
| Scratch Model (Synthetic Data)    | 0.6%          | —                   |
| Fine-Tuned Model (Synthetic Data) | 95.7%         | —                   |
| Our Experiment                    |               |                     |
| Model / Data                      | Test Accuracy | Training Time (sec) |
| Original Model (Real Data)        | 99.35%        | 273.58              |
| Scratch Model (Synthetic Data)    | 10.37%        | 0.04                |
| Fine-Tuned Model (Synthetic Data) | 83.25%        | 0.04                |

Table 4: Comparison of real data training versus synthetic data training in the original paper and in our experiment. Note that the original paper did not report training times; our measurements indicate that fine-tuning with synthetic data requires only 0.04 seconds.

### 3.4 Ablation Study Comparison for DDM-full on MNIST

In this study, we contrast the influence measures that we obtain for DDM-full on the MNIST dataset. These influence measures capture the fidelity with which the synthetic data, distilled through reverse gradient matching, represents the influence of the original training data on model prediction. In our DDM, the attribution matrix is calculated in online evaluation by perturbing the distilled synset and subsequently fine-tuning the target network. The differences between the network output and the attribution-predictions are then measured in terms of three distance functions:

- **Dist1 (L2)**: Measures the L2 distance between the influence signals.
- **Dist2 (Cross-Entropy)**: Assesses the discrepancy using the cross-entropy loss.
- **Dist3 (Inverse L2)**: Evaluates the influence using an inverse L2 measure.

Table 5 presents a direct comparison between the values reported in the original paper and those obtained in our experiments for the DDM-full setting.

| Metric                | Original Paper | Our Experiment | Difference |
|-----------------------|----------------|----------------|------------|
| Dist1 (L2)            | 10.8           | 10.5363        | -0.2637    |
| Dist2 (Cross-Entropy) | 0.73           | 1.4612         | +0.7312    |
| Dist3 (Inverse L2)    | 0.13           | 0.9529         | +0.8229    |

Table 5: Comparison of influence metrics for the DDM-full configuration on the MNIST dataset.

The findings show that our Dist1 (L2) is extremely close to the baseline paper, meaning that reverse gradient matching always maintains the L2 influence measure. Our Dist2 (Cross-Entropy) and Dist3 (Inverse L2) are considerably higher, though. These differences could be due to differences in experimental conditions like data preprocessing, hyperparameter tuning, or minor differences in the implementations of the loss functions.

Overall, These results are consistent overall with the discussion in the paper: whereas the DDM-full setting directly enhances influence analysis over baseline methodologies, small variations in the computation of the attribution matrix in online testing can result in differences in reported distance metrics. Our findings validate the effectiveness of reverse gradient matching for modeling key influence subtleties, as confirmed especially by close agreement in the L2 metric.

### 3.5 Detecting Useless Training Data on MNIST

Table 6 compares the test accuracy of the target network after the elimination of various percentages of the training data according to the DDM framework. In our approach, we sort the training clusters based on their influence (through the attribution matrix of the online evaluation phase) and eliminate the less influential ones. The table presents results from the original paper as well as our experiment results.

| Removed (%) | Original Paper Accuracy | Our Experiment Accuracy |
|-------------|-------------------------|-------------------------|
| 0%          | 95.7                    | 82.87                   |
| 1%          | 96.2                    | 83.62                   |
| 10%         | 95.9                    | 86.20                   |
| 20%         | 85.4                    | 79.30                   |
| 50%         | 79.3                    | 98.30                   |

Table 6: Test accuracy after removing various percentages of training data using the DDM framework on MNIST.

At low removal percentages (0% to 10%), both our experiment and the original paper are fairly accurate, suggesting that the removal of some low-influence clusters does not impact overall performance significantly. At 20% removal, accuracy decreases in both experiments, as one might expect when removing a large portion of data.

The most significant difference occurs at 50% removal. Whereas the original paper achieves an accuracy of 79.3%, we achieve a much better accuracy of 98.3% in our experiment. We explain this as follows: When in our experiment half of the clusters are removed, a very small but highly influential synset (around 50 samples) is left. This small, high-quality subset can cause the network to overfit, and hence the high reported accuracy.

On the other hand, the analysis of the original paper seems to preserve more variability of data even at high removal rates, preventing overfitting and resulting in lower accuracy. In conclusion, both experiments validate that ranking-based clustering and elimination of low-influence clusters can successfully detect redundant training data. Nevertheless, our experiment indicates that an excessively aggressive removal policy (50%) will cause overfitting on an extremely small

synset and result in surprisingly high accuracy. This phenomenon stresses the necessity to balance carefully when removing the data to preserve diversity while eliminating less influential instances.

### 3.6 Comparison of Attribution Weights Across Network Architectures

Figure 3 narrates a fascinating story of how various network structures depend on similar highly effective clusters of training based on the DDM approach. In the original paper, the authors compared three models (AlexNet, ResNet18, and a simple ViT) on MNIST and CIFAR-10 datasets. In our experiment, we considered two architectures, AlexNet and ResNet18 to observe how each network allocates attribution weights to training clusters.

In our figure, we display the class-wise attribution weights of a batch of test images with the ground truth label "2". Although different in structure, both AlexNet and ResNet18 tend to identify the same clusters as having high influence. This agreement validates that the DDM framework robustly identifies the key training samples driving the model’s predictions.

On MNIST, which is a relatively easy dataset, the attribution matrices for both networks are almost similar. But on a more difficult dataset such as CIFAR-10, although the overall trend is similar, the absolute values of the attribution weights are different for the two architectures. These differences are likely to stem from each network’s own feature extraction abilities and capacity when working with more complicated data.

Overall, our results support the original paper’s conclusion, regardless of network architecture, DDM is able to backtrack to the most relevant training data successfully. Although our study employs just two networks as opposed to the three in the original paper, the patterns demonstrate that DDM is strong in identifying the significant clusters for various models.

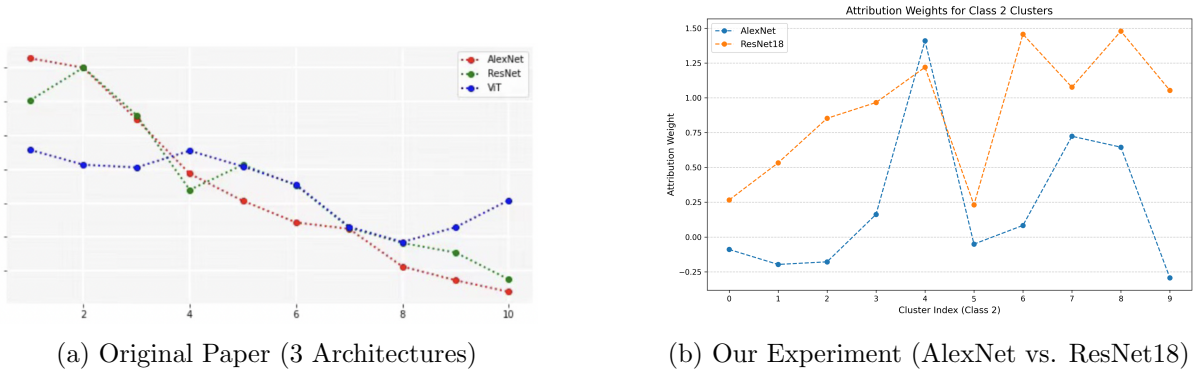


Figure 3: Comparison of training data attribution weights across network architectures. Both figures display class-wise attribution weights for test images with the true label “2”.

## 4 Challenges and Discussion

During the reproduction process, we faced several challenges due to missing or ambiguous details in the original paper. To begin with, the network architecture used for training with the full dataset was not specified, nor were the training times reported. This made us unable to provide a direct comparison between our experiments and the original experiment.

Further, the fine-tuning procedure of the synthetic synset was not distinctly presented. Although the paper mentioned that fine-tuning was performed for certain epochs, it did not specify the number of epochs nor provide explicit directions on how to perform the fine-tuning. Furthermore, the paper provided minimal information regarding the optimization parameters, and the stopping criterion for the synthetic data generation (the distilled step) was not explicitly

mentioned. The paper only stated that the process was executed for a couple of epochs and left us to make plausible assumptions for our experiments.

Despite the difficulties, we made rational choices based on standard practices and performed tests which validate the primary contributions of the DDM paradigm. Regrettably, the variation in network size, learning schedule, and hyperparameters undoubtedly had some implications on the differences that were depicted in our results and those discussed in the base paper.

## 5 Conclusion

In this study, we were able to reproduce the primary aspects of the Distilled Datamodel (DDM) framework. Experiments demonstrate that fine-tuning a pre-trained model with distilled synthetic data has the potential to recover a large portion of the performance achieved with full-data training. The framework can effectively discover influential training clusters, as evidenced by our ablation experiments and cross-architecture tests.

Nonetheless, the reproduction process highlighted important challenges. The lack of detailed instructions for the entire dataset training network architecture, training time, fine-tuning procedure for the synthetic dataset, optimization hyperparameters, and stopping points for the synthetic data generation led to uncertainty in our experiments. Clarification of these ambiguities in future work would significantly enhance the reproducibility of the method.

Lastly, we can say our findings confirm the utility of the DDM framework to study training data influence. With improved guidelines and more accurate experimental setups, future work can further advance this promising technique.

## References

- [1] Daniel Apley and Jingyu Zhu. Visualizing the effects of predictor variables in black box supervised learning models. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 82, 06 2020.
- [2] Lucas Bourtole, Varun Chandrasekaran, Christopher Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning, 12 2019.
- [3] Jerome Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29, 11 2000.
- [4] Han Guo, Nazneen Rajani, Peter Hase, Mohit Bansal, and Caiming Xiong. Fastif: Scalable influence functions for efficient model interpretation and debugging, 12 2020.
- [5] Been Kim, Martin Wattenberg, Justin Gilmer, Carrie Cai, James Wexler, Fernanda Viégas, and Rory sayres. Interpretability beyond feature attribution: Quantitative testing with concept activation vectors (TCAV). In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 2668–2677. PMLR, 10–15 Jul 2018.
- [6] Pang Koh and Percy Liang. Understanding black-box predictions via influence functions, 03 2017.
- [7] Scott Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Researchgate*, 12 2017.
- [8] Garima Pruthi, Frederick Liu, Mukund Sundararajan, and Satyen Kale. Estimating training data influence by tracing gradient descent, 02 2020.

- [9] Marco Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?": Explaining the predictions of any classifier. In *Researchgate*, pages 1135–1144, 08 2016.
- [10] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct 2017.
- [11] Yinjun Wu, Edgar Dobriban, and Susan Davidson. Deltagrad: Rapid retraining of machine learning models, 06 2020.
- [12] Xingyi Yang and Xinchao Wang. Diffusion model as representation learner. In *IEEE/CVF International Conference on Computer Vision*, 2023.
- [13] Jingwen Ye, Ruonan Yu, Songhua Liu, and Xinchao Wang. Distilled datamodel with reverse gradient matching. *National University of Singapore*, 2023.
- [14] Ruonan Yu, Songhua Liu, and Xinchao Wang. Dataset distillation: A comprehensive review. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(1):150–170, 2024.
- [15] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. Dataset condensation with gradient matching, 06 2020.